

# Building a Model

## Gathering Data

# Data Sets

Every model is only as good as the data it uses

There are many existing data sets that you can use to train your language model

- [WebText2](#) - All of Reddit up to April 2020
- [Bookcorpus](#) - 1.1GB of eBook text
- [Library Genesis](#) - 35GB of digitized book text
- [HuggingFace](#) - Open source AI site with links to many collections
- [CommonCrawl](#) - Free open repository of web crawl data

# Scrape Your Own Data!

Existing datasets come with a lot of issues

- Incorrect facts
  - Outdated, unverified, purposeful poisoning
- Specific word usages
  - Slang, regional terms
- Low-resource languages
  - If not English, Japanese, Spanish, Chinese...
- Innate Biases
- Noise

# Roll Your Own Language Model!

## Web Scraping 101

### 1. Identify the source web sites

- What are you trying to do with your data?
- What are you looking for?
- What **aren't** you looking for?
- What medium(s)?
- Does age of data matter?
- Dynamic vs static data?

# Roll Your Own Language Model!

## 2. Choose a scraping tool

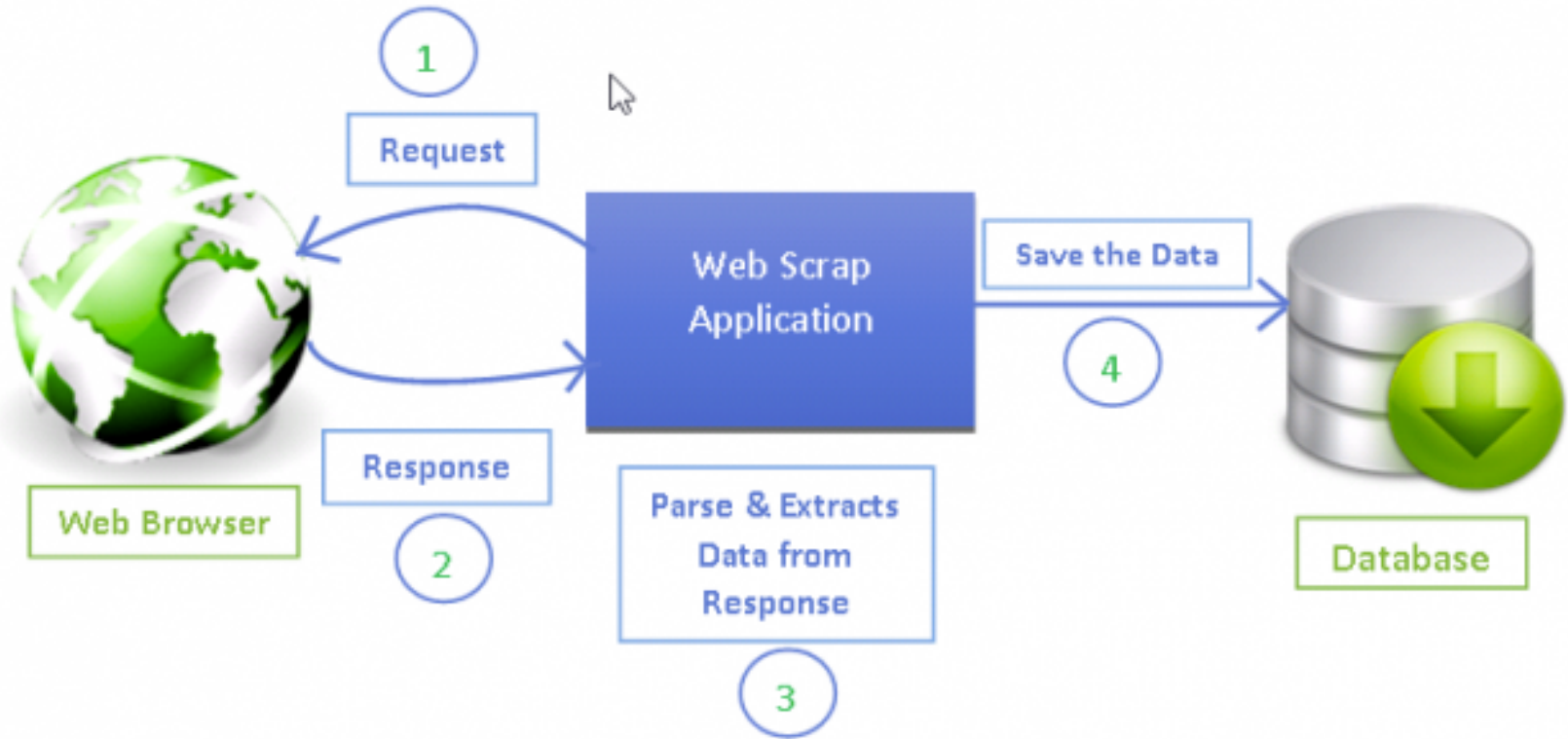
- [BeautifulSoup \(python\)](#)
- [ScraPy \(python\)](#)
- [Selenium \(various\)](#)
- [JSoup \(java\)](#)
- [HTMLUnit \(java\)](#)
- [Tons of JavaScript options](#)
- [CPR \(C++\)](#)
- [Scalpel \(Haskell\)](#)

# Roll Your Own Language Model!

## 3. Write a script

- Target the type of data you want
- URLs, HTML tags, keywords, file types, etc
  - Most web text content is contained within `<p>` and `<h#>` tags
  - Modern responsive pages bury the content
  - Many sites store text in PDF files
- Dynamic Crawling?

# Roll Your Own Language Model!



# Roll Your Own Language Model!

4. Run the script - This will take time (i.e. days)

- Do not let it get too greedy (timeouts, cooldowns)
- Respect [robots.txt](#) files
- Only scrape public data



# Roll Your Own Language Model!

## 5. Clean and preprocess the data

- normalizing
- tokenizing
- de-duping
- remove HTML tags, etc
- **SAVE YOUR RAW DATA**

## 6. Train your LM

# 1. Identify Sources

- The most important part of building a LM
- Identify the task you are trying to implement
- Focus on sources that are directly related first
  - Indirect relationships are important for breadth, and will come naturally
- Make liberal use of keywords, tags, links, and other identifying features when discovering sources

## 2. Scraping Tools

- Different tools offer different features
- BeautifulSoup, Scrapy, JSoup are simple parsers
  - Downloads pages and extracts data
- Selenium is a collection of tools that automates web browsers
  - For more complex tasks
  - Much better for non-text scraping



- Common Crawl is slightly different
- It is both a scraping tool and a repository
- Archives of 15 years of web scraping
  - Literal Petabytes of TEXT
- Archives themselves can be searched and scraped
- [Tons of good tutorials](#)

### 3. Write Your Script

- Script should do a few basic things
  - Identify a starting URL or list of starting URLs
  - Set the search parameters
    - What is it looking for
    - What should it explicitly ignore
  - Check for a robots.txt file
    - Use the contents to limit the links to follow
  - Parse the page and save the data

# Example script in BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

# Send a request to the website
url = 'https://www.example.com'
response = requests.get(url)

# Parse the HTML content using
BeautifulSoup
soup =
BeautifulSoup(response.content,
'html.parser')
```

```
# Extract the data you need
title = soup.find('title').text
paragraphs = soup.find_all('p')
links = soup.find_all('a')

# Print the data
print('Title:', title)
print('Paragraphs:')
for p in paragraphs:
    print(p.text)
print('Links:')
for link in links:
    print(link.get('href'))
```

## 4. Running Your Script

- Scripts are very fast compared to people
  - Often too fast
  - Include "cooldowns"
  - Queue unresponsive links
- BE CAREFUL WHAT YOU SCRAPE!
  - Pay attention to the Robots.txt file
  - Do not follow any links not whitelisted
  - [And don't tell Missouri what you find](#)

## 4a. Legal/Ethical

- Most authorities do not understand what you're doing
- Cannot stop you from doing it
  - Therefore they are afraid of it
- Very few laws have been passed
  - Because it's hard to figure out what is illegal about it
- Lawsuits are starting





## 4a. Legal/Ethical

- What should you worry about?
  - Are you monetizing this?
  - Who "owns" the text?
  - What is the purpose of the text?
  - Has the owner communicated their wishes?
  - Where will this be used?
  - How will this be used?



## 5. Clean and Preprocess Your Data

- Scraped data is a mess
  - HTML tags, spacing, page numbers, headers, footers, image captions, duplicate pages, etc, etc
- More scripts are needed to put data into a usable form
- Tokenizing happens here, along with N-Gram generation
- Filtering out unwanted terms or documents

## 5. Clean and Preprocess Your Data

- Be sure to anonymize your data before processing
  - Any identifying content that is not a public figure should be removed if not needed
    - SSN, ID#, dates, account names



## 6. Train the Language Model

- Once the data is normalized, you can run the math
- Token and/or N-Gram counting
- POS Tagging
- Smoothing/Interpolation
- Joint Probabilities
- Embeddings
- Much, much more...

# Building a Model

## Gathering Data